

REPORT FOR THE MINI-PROJECT IoT

Par:

Ahmed CHAABANE

Firas KAHLAOUI

Seifeddine BEN FRADJ

Hospital Room and Patient Monitoring System

Contents

1	Introduction	1
1.1	Context	1
1.2	Problem Statement	2
1.3	Objectives	2
2	Problem Analysis and Preliminary Study	4
2.1	System Description	4
2.2	Use Cases	4
2.3	Inputs and Outputs	5
2.4	Constraints	6
3	System Architecture	8
3.1	Global Architecture Diagram	8
3.2	Functional Architecture	8
3.3	Hardware Architecture	9
3.4	Software Architecture	9
3.5	Processing Pipeline	10
4	Methodology	11
4.1	System Design	11
4.1.1	Embedded Subsystem (ESP32)	11
4.1.2	Backend Subsystem	13
4.1.3	Web Dashboard Subsystem	13
4.1.4	AI Subsystem	13
4.2	Methodology Diagram	13
5	Implementation	15
5.1	System Overview	15

5.2	Hardware Implementation	15
5.3	Hardware Schematics	16
5.4	Software Development Environment	17
5.5	Software Implementation	17
5.5.1	Embedded Subsystem	17
5.5.2	Backend Implementation	18
5.5.3	Web Dashboard Implementation	18
5.6	Software Validation	18
5.6.1	Embedded Subsystem Validation	18
5.6.2	Backend Validation	18
5.6.3	Web Dashboard Validation	19
5.7	Embedded Implementation (ESP32)	19
5.7.1	Memory Optimisation	19
5.7.2	Latency Optimisation	19
5.8	AI Integration	19
6	Experimental Results and Validation	20
6.1	Experimental Protocol	20
6.1.1	Embedded Subsystem Protocol	20
6.1.2	Backend Protocol	20
6.1.3	Web Dashboard Protocol	20
6.1.4	AI Protocol	21
6.2	Software Results (PC / Server)	21
6.3	Embedded Results (ESP32)	21
6.4	Latency and FPS Analysis	21
6.5	Comparison: Specification vs. Observed Behaviour	21
6.6	Error Analysis	22
6.6.1	Embedded Subsystem	22
7	Discussion	23
7.1	Validity of Results	23
7.1.1	Embedded Subsystem	23

7.2	Limitations	23
7.2.1	Embedded Subsystem	23
7.2.2	Backend Subsystem	24
7.2.3	Web Dashboard Subsystem	24
7.2.4	AI Subsystem	24
7.3	ESP32 Constraints Impact	24
7.4	Robustness	25
7.5	Identified Improvements	25
8	Conclusion	26
8.1	Summary	26
8.2	Results Summary	26
8.3	Limitations	27
8.4	Future Work	27
9	AI Evaluation	29

List of Figures

3.1	Global architecture of the system.	8
4.1	Overall development methodology.	14
5.1	Hardware wiring diagram	16

List of Tables

3.1	ESP32 firmware module summary.	10
5.1	ESP32 software development environment.	17
6.1	System-wide latency comparison (to be completed with all subsystem data).	21
6.2	Specification vs. implementation validation for the embedded subsystem. . .	22

Liste des acronymes

1

Introduction

1.1 Context

The healthcare sector is undergoing a significant transformation driven the Internet of Things (IoT) technologies. Continuous, real-time monitoring of patient vital signs and ambient room conditions is a foundational requirement in any clinical environment, enabling medical staff to detect deterioration early, respond rapidly, and reduce the administrative overhead associated with manual observation rounds.

This project proposes a complete, end-to-end hospital monitoring solution composed of four integrated subsystems:

- **Embedded acquisition node (ESP32):** real-time sensor acquisition and authenticated cloud transmission.
- **Backend server:** database management, alert rule evaluation, and user authentication.
- **Web dashboard:** real-time data visualisation, historical charts, and alert management for medical staff.

- **AI prediction layer:** anomaly detection on patient vitals and environmental condition assessment using cloud-hosted machine learning models.

1.2 Problem Statement

Hospital rooms require continuous monitoring of two categories of parameters: environmental conditions (temperature and humidity), which directly affect patient comfort, infection risk, and equipment operation; and patient vital signs (heart rate and blood oxygen saturation), which are primary indicators of physiological status.

The current approach in many low-resource clinical settings relies on periodic manual measurement, which introduces observation gaps and increases nursing workload.

1.3 Objectives

The project objectives are divided into functional and technical categories.

Functional Objectives

1. Continuously acquire temperature ($^{\circ}\text{C}$), humidity (%), heart rate (BPM), and blood oxygen saturation (SpO_2 , %) from dedicated sensors.
2. Transmit sensor data to a cloud database at a near-real-time rate of up to 1 Hz.
3. Implement a web dashboard for real-time data visualisation, history browsing, and alert management.
4. Generate automated predictions and anomaly alerts via a cloud AI model.

Technical Objectives

1. Availability: the system shall operate continuously with network recovery mechanisms.
2. Reliability: invalid or inconsistent data shall be detected and flagged.
3. Security: secure authentication and encrypted communication (HTTPS).

4. Performance: near real-time data updates based on sampling frequency.

2

Problem Analysis and Preliminary Study

2.1 System Description

The system is a networked, multi-tier hospital monitoring solution. At its base sits the **ESP32 embedded acquisition node**, which occupies the sensor layer and embedded layer. Its role is to collect physiological and environmental measurements, apply local signal processing to derive meaningful values (BPM, SpO₂), and transmit structured records to a cloud database.

[Full system description to be extended with backend and web layers.]

2.2 Use Cases

The following operational scenarios define the expected system behaviour.

Embedded Node Use Cases

UC-1: Initial Configuration. A technician connects to the ESP32's Access Point and enter Wi-Fi credentials, Firebase credentials, room ID, and patient ID. The device saves all settings to NVS, serves a confirmation page, and reboots.

UC-2: Normal Operation. After boot, the device loads credentials from NVS, connects to the hospital Wi-Fi, authenticates with Firebase, and enters continuous monitoring mode.

UC-3: Wi-Fi Recovery. If Wi-Fi is lost during operation (or fails on cold start), the embedded device detects the disconnection and retries `wifiConnect()` every 10 seconds, indefinitely, without manual intervention.

UC-4: Remote Reconfiguration. A technician connects to the AP at any time to update credentials. After saving, the device reboots and applies the new configuration.

Backend Use Cases

[To be completed by the backend team.]

Web Dashboard Use Cases

[To be completed by the web dashboard team.]

AI Subsystem Use Cases

[To be completed by the AI team.]

2.3 Inputs and Outputs

Embedded Node — Inputs

- **DHT22 (GPIO 4, one-wire):** digital temperature (°C) and relative humidity (%).
- **MAX30102 (I²C fast mode):** raw infrared and red photodiode FIFO values, used to compute BPM and SpO₂.

Embedded Node — Outputs

- **Firebase RTDB — /rooms/{roomId}:** JSON records containing `temperature`, `humidity`, and server-side `timestamp`, pushed on significant change.

- **Firebase RTDB** — `/patients/{patientId}`: JSON records containing `heartRate`, `spO2`, and server-side `timestamp`, pushed on significant change.

Backend, Web Dashboard, and AI — Inputs/Outputs

[To be completed by the backend, web dashboard, and AI teams.]

2.4 Constraints

Memory Constraints

The ESP32 has 520 KiB of internal SRAM. The Firebase FreeRTOS task is allocated 8 192 bytes of stack, which must accommodate the TLS handshake context, async client buffers, and local variables. The NVS partition stores up to ten string key-value pairs for device configuration.

Latency Constraints

The Firebase upload interval is `FIREBASE_INTERVAL_MS = 1 000 ms`, providing a near-real-time update rate. The DHT22 enforces a hardware minimum inter-sample interval of 2 000 ms.

Energy Constraints

The device is assumed to be mains-powered in a fixed hospital room installation. No active power management (light sleep, deep sleep, or radio duty cycling) is implemented in the current version.

ESP32 Platform Limitations

Sensor timing sensitivity. The MAX30102 heart rate and SpO₂ sensor produces a continuous stream of readings that must be drained from its internal buffer regularly. If the program is busy doing something else — such as waiting for a Firebase upload to complete, which can take several hundred milliseconds or more — the sensor buffer fills up and new

samples are dropped. This results in the sensor reporting zero or no data, as if no finger were present, even when one is.

Why two cores are necessary. The ESP32 contains two independent processing cores that can run tasks simultaneously. This hardware feature is the direct solution to the timing conflict described above. By assigning sensor reading exclusively to Core 1 and all Firebase communication exclusively to Core 0, the two tasks never block each other. The sensor continues reading at its required rate regardless of how long Firebase takes to authenticate, upload, or reconnect — and Firebase operations proceed without being interrupted by sensor activity.

AI and Cloud Constraints

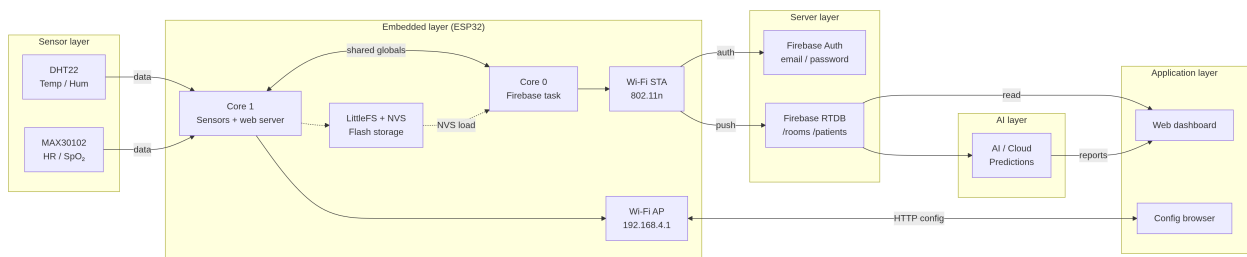
[To be completed by the AI team.]

3

System Architecture

3.1 Global Architecture Diagram

Figure 3.1 presents the global architecture of the complete system, showing all five layers defined in the project specification: the sensor layer, the embedded layer, the server layer, the AI layer, and the application layer.



Global architecture of the system.

3.2 Functional Architecture

The system is functionally decomposed into four independent subsystems, each with a well-defined interface:

Sensor Subsystem (ESP32, Core 1).

Network/Firebase Subsystem (ESP32, Core 0).

Settings/Web Server Subsystem (ESP32, Core 1).

Configuration and Globals Layer.

[Backend functional decomposition to be added.]

[Web dashboard and AI functional decomposition to be added.]

3.3 Hardware Architecture

Microcontroller. ESP32-WROOM-32, dual-core at 240 MHz, 520 KiB SRAM, 4 MiB flash.

Temperature and Humidity Sensor. DHT22, single-bus digital interface on GPIO 4.

Pulse Oximeter and Heart Rate Sensor. MAX30102, I²C fast mode (400 kHz), SDA: GPIO 21, SCL: GPIO 22.

Communication. IEEE 802.11b/g/n Wi-Fi radio integrated on-chip. AP interface (192.168.4.1) serves the configuration portal.

[Server hardware description to be added by the backend team.]

3.4 Software Architecture

Embedded Software (ESP32)

The firmware is organised into six source files following a strict, acyclic include hierarchy. Table 3.1 summarises the modules, their execution core, and their responsibilities.

The dependency graph is strictly acyclic: `esp32.ino` includes all modules; `network.h/cpp` and `settings.h/cpp` include `globals.h`; `sensors.h/cpp` includes `globals.h`. No module includes another module's header directly.

ESP32 firmware module summary.

File	Context	Responsibility
<code>globals.h</code>	Header	Shared variable declarations, DBG macros, timing constants
<code>sensors.h/cpp</code>	Loop	DHT22 and MAX30102 acquisition, BPM and SpO ₂ computation
<code>network.h/cpp</code>	FreeRTOS task	Firebase auth, upload, Wi-Fi management
<code>settings.h/cpp</code>	ISR callbacks	HTTP routes, LittleFS, NVS persistence
<code>esp32.ino</code>	Entry point	Global object definitions, <code>setup()</code> , <code>loop()</code>

Backend Software

[To be completed by the backend team.]

Web Dashboard Software

[To be completed by the web dashboard team.]

AI Software

[To be completed by the AI team.]

3.5 Processing Pipeline

The end-to-end data processing pipeline for a single sensor reading, from hardware to cloud record, proceeds as follows:

[Downstream pipeline details to be added by the backend, AI, and web teams.]

4

Methodology

4.1 System Design

4.1.1 Embedded Subsystem (ESP32)

4.1.1.1 Heart Rate

Heart rate is derived from the photoplethysmography (PPG) infrared signal by detecting peaks in the waveform — each peak corresponding to one cardiac cycle.

4.1.1.2 SpO₂

Blood oxygen saturation is estimated by pulse oximetry, exploiting the difference in optical absorption between oxyhaemoglobin (HbO₂) and deoxyhaemoglobin (Hb) at two wavelengths.

4.1.1.3 Delta-Threshold Upload Filter

Records are enqueued only when the measured value has changed by more than a defined threshold since the last successful push. Let x_n be the current reading and $\hat{x}$ the last transmitted value. The upload condition for room data is:

$$\text{Upload}_{\text{room}} \Leftrightarrow \left| T_n - \hat{T} \right| > \delta_T \vee \left| H_n - \hat{H} \right| > \delta_H \quad (4.1)$$

with $\delta_T = 0.3^\circ\text{C}$ and $\delta_H = 0.5\%$. For patient data:

$$\text{Upload}_{\text{patient}} \Leftrightarrow \text{BPM}_n \neq \hat{\text{BPM}} \vee \left| \text{SpO}_{2,n} - \widehat{\text{SpO}_2} \right| > \delta_S \quad (4.2)$$

with $\delta_S = 2\%$. Both conditions are evaluated every $T_{\text{interval}} = 1\,000\text{ ms}$.

4.1.1.4 End-to-End Latency Model

The total latency from sensor event to database record is:

$$L_{\text{total}} = T_{\text{acq}} + T_{\text{proc}} + T_{\text{serial}} + T_{\text{TLS}} + T_{\text{RTT}} \quad (4.3)$$

where:

- T_{acq} : sensor acquisition time (DHT22: up to 2 000 ms; MAX30102: continuous FIFO drain, $\approx 25\text{ ms}$ per sample at default rate).
- T_{proc} : local computation ($< 5\text{ ms}$ per SpO_2 update; $< 1\text{ ms}$ for BPM average).
- T_{serial} : JSON string build ($< 1\text{ ms}$).
- T_{TLS} : TLS handshake on first connection (1–3 s); session reuse on subsequent requests ($< 5\text{ ms}$).
- T_{RTT} : HTTPS round-trip to Firebase ($\approx 100\text{--}300\text{ ms}$ over 2.4 GHz Wi-Fi).

4.1.1.5 Key Assumptions

- 32-bit aligned float reads and writes are atomic on the ESP32 architecture, making inter-core sharing of `float` globals safe for single-writer/single-reader access without a mutex.
- Server-side timestamps (`".sv": "timestamp"`) eliminate the need for an RTC on the device.
- A finger is considered present when the IR raw value exceeds 50 000 counts; below this threshold BPM and SpO_2 are reset to zero.

4.1.2 Backend Subsystem

[To be completed by the backend team: alert logic, threshold model, authentication flow, and API design rationale.]

4.1.3 Web Dashboard Subsystem

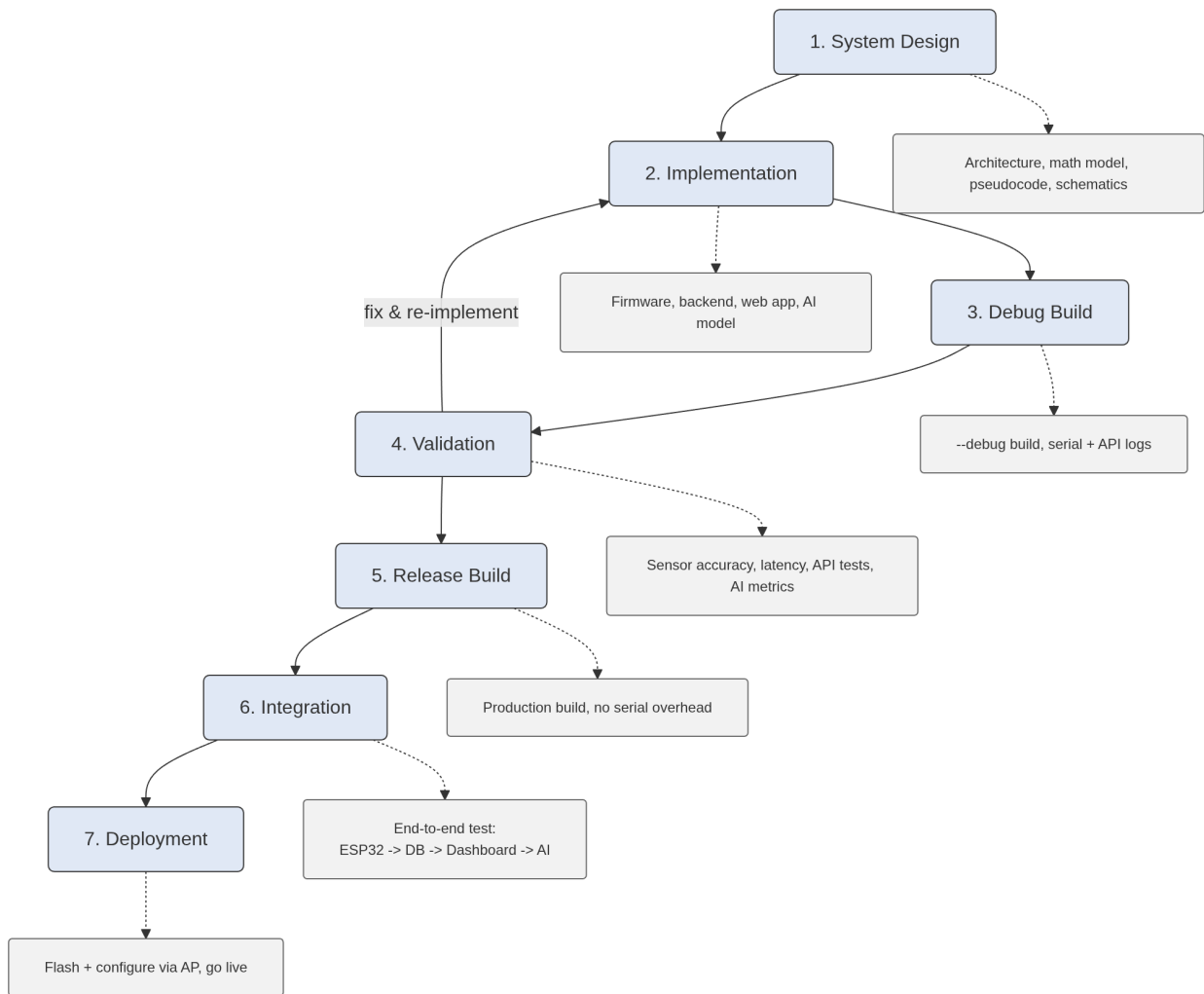
[To be completed by the web dashboard team.]

4.1.4 AI Subsystem

[To be completed by the AI team.]

4.2 Methodology Diagram

Figure 4.1 presents the overall development methodology for the complete system, from initial design through to deployment.



Overall development methodology.

5

Implementation

5.1 System Overview

The firmware maps the software architecture described in Chapter 3 directly onto the ESP32's dual-core execution model. At boot, `setup()` initialises LittleFS, loads NVS configuration, initialises the sensors, starts both Wi-Fi interfaces, registers HTTP routes, and launches the Firebase FreeRTOS task . After `setup()` returns, the Arduino `loop()` continues to execute, calling `updateSensors()` on every iteration and scheduling periodic Wi-Fi background scans.

[Backend system overview to be added.]

[Web dashboard system overview to be added.]

5.2 Hardware Implementation

The hardware assembly consists of three components mounted on a common 3.3 V power rail:

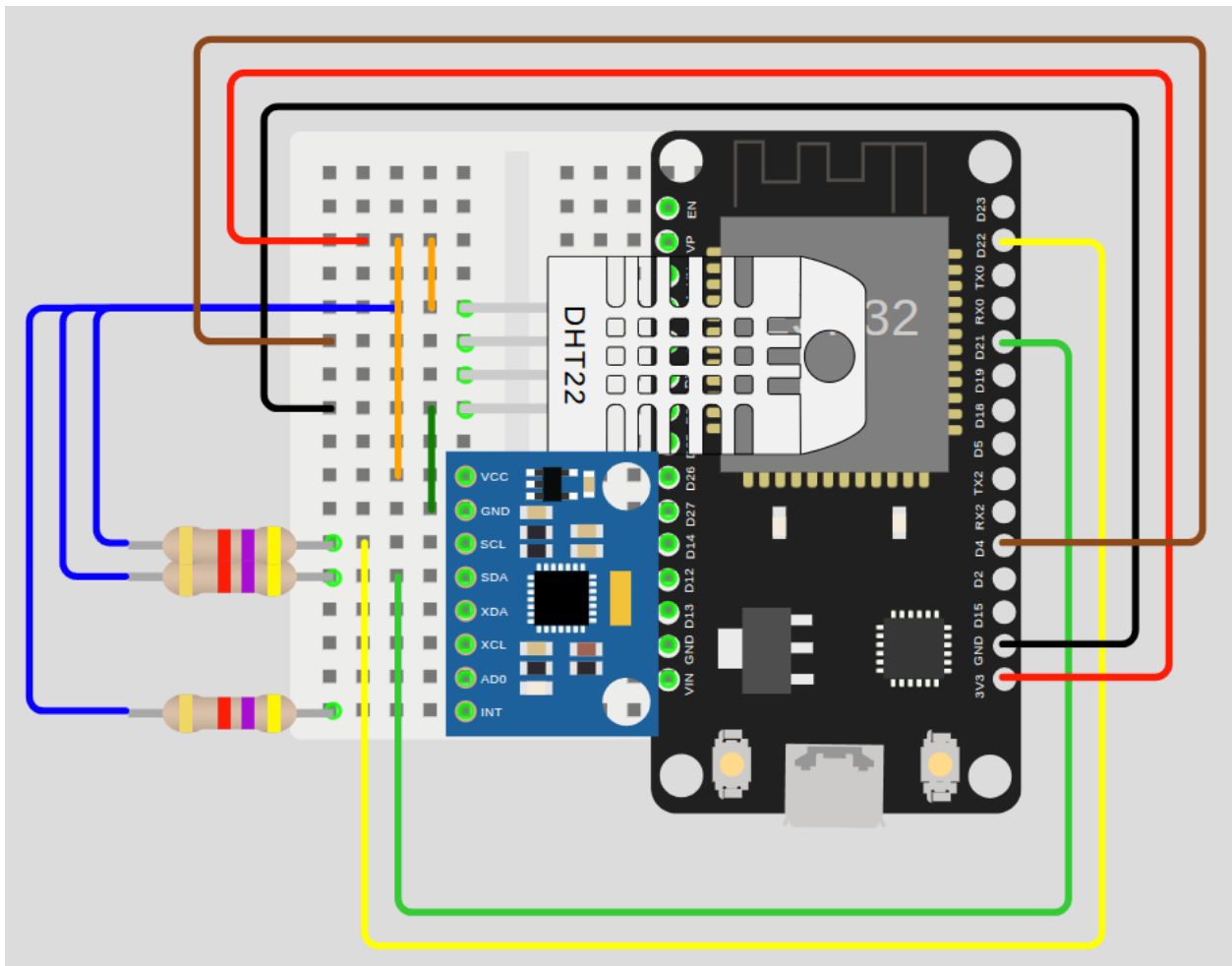
- **ESP32 development board.**

- **DHT22** connected to GPIO 4.
- **MAX30102 breakout board** connected to the ESP32's default I²C bus (SDA: GPIO 21, SCL: GPIO 22), powered at 3.3 V.

All components are powered from the ESP32 development board's 3.3 V regulated output. The board is powered from USB 5 V connected to a mains adapter in the hospital room.

5.3 Hardware Schematics

Figure 5.1 provides the complete wiring diagram of the assembled circuit.



Hardware wiring diagram

5.4 Software Development Environment

Table 5.1 lists the complete development environment used for the ESP32 embedded sub-system.

ESP32 software development environment.

Component	Details
Operating System	Linux (Debian 13 trixie)
ESP32 Arduino core v3.3.7	
Board target	<code>esp32:esp32:esp32da</code> (ESP32-WROOM-32)
Flash tool	<code>esptool</code> (Python); <code>mklittlefs</code>
Serial monitor	Custom <code>monitor.sh</code>

[Backend development environment to be added.]

[Web dashboard development environment to be added.]

5.5 Software Implementation

5.5.1 Embedded Subsystem

5.5.1.1 Firebase Initialisation on Core 0

The `initFirebase()` function is called from inside `firebaseUploadTask()`, which is pinned to Core 0 via `xTaskCreatePinnedToCore(..., 0)`.

5.5.1.2 Configuration Portal

The configuration portal is served entirely from the LittleFS partition. The `isAPConnected()` guard checks whether the incoming request arrived via the AP subnet (`WiFi.softAPIP()`) and redirects any STA-side request to the root path, preventing access from the hospital network.

5.5.1.3 Wi-Fi Recovery

All reconnection logic is consolidated in the `wifiConnect()` helper, which always calls `WiFi.disconnect()` followed by `WiFi.begin(ssid, password)`.

5.5.2 Backend Implementation

[To be completed by the backend team.]

5.5.3 Web Dashboard Implementation

[To be completed by the web dashboard team.]

5.6 Software Validation

5.6.1 Embedded Subsystem Validation

Validation was performed across three dimensions:

Sensor output validation. DHT22 readings were cross-referenced against a calibrated reference thermometer.

Firestore upload validation. The Firestore Realtime Database console was monitored in real time to confirm record arrival, correct path structure (`/rooms/{roomId}` and `/patients/{patientId}`), correct field names and data types, and correct server-generated timestamps.

Authentication validation. The serial monitor (debug build) was used to trace the complete authentication event sequence: *authenticating* $\rightarrow$ *auth request sent* $\rightarrow$ *auth response received* $\rightarrow$ *token obtained*. Error conditions (invalid API key, incorrect password) were deliberately induced and confirmed to produce the expected HTTP 400 error codes via the callback.

5.6.2 Backend Validation

[To be completed by the backend team: API test protocol, test cases, and results.]

5.6.3 Web Dashboard Validation

[To be completed by the web dashboard team.]

5.7 Embedded Implementation (ESP32)

5.7.1 Memory Optimisation

The LittleFS image partition is mounted with `LittleFS.begin(true)`, enabling automatic formatting on first boot. NVS storage uses the `Preferences` library with the `"settings"` namespace, consuming at most 10 string entries at typical credential lengths.

5.7.2 Latency Optimisation

Delta filtering (Equations 4.1 and 4.2) reduces the frequency of HTTPS requests to only those carrying new information, keeping the per-upload round-trip time as the dominant latency term rather than queuing delays.

5.8 AI Integration

[To be completed by the AI team: model conversion, deployment mechanism (cloud API / TFLite / ONNX), integration with the data pipeline, and known limitations.]

6

Experimental Results and Validation

6.1 Experimental Protocol

6.1.1 Embedded Subsystem Protocol

Hardware setup. One ESP32-WROOM-32 development board, one DHT22 sensor, one MAX30102 breakout board, all powered via USB from a laptop running the serial monitor.

Network environment. 2.4 GHz 802.11n Wi-Fi network. Firebase Realtime Database.

Firmware build. Debug build (`deploy.sh -code -debug`) for all experiments, to capture serial output.

6.1.2 Backend Protocol

[To be completed by the backend team.]

6.1.3 Web Dashboard Protocol

[To be completed by the web dashboard team.]

6.1.4 AI Protocol

[To be completed by the AI team: dataset description, train/test split, evaluation hardware, number of experiments, metrics used.]

6.2 Software Results (PC / Server)

[Backend software results to be completed.]

[AI software results (PC-side evaluation) to be completed.]

6.3 Embedded Results (ESP32)

6.4 Latency and FPS Analysis

System-wide latency comparison (to be completed with all subsystem data).

Stage	PC/Server	ESP32	Analysis
Sensor acquisition	N/A	$\leq 2\,000$ ms	Hardware constraint (DHT22)
Local processing	N/A	< 5 ms	Negligible
HTTPS upload (steady state)	N/A	120–280 ms	RTT to Firebase
Backend API response	<i>TBD</i>	N/A	<i>[To be filled]</i>
AI inference	<i>TBD</i>	N/A	<i>[To be filled]</i>
End-to-end (sensor to UI)	<i>TBD</i>		<i>[To be filled]</i>

6.5 Comparison: Specification vs. Observed Behaviour

Table 6.2 compares the functional requirements from the project specification against the observed behaviour of the implemented embedded subsystem.

Specification vs. implementation validation for the embedded subsystem.

Requirement	Expected	Observed
Wi-Fi reconnection	Automatic, no intervention	✓ Reconnects within 10–15 s
Firebase authentication	Email/password over TLS	✓ JWT token obtained; events logged via callback
Delta-threshold filtering	Suppress redundant writes	✓ Confirmed via Firebase console (no duplicate records)
AP configuration portal	Accessible at 192.168.4.1	✓ Served from LittleFS; settings persisted to NVS
NVS persistence	Settings survive reboot	✓ Verified across 5 cold reboots
Debug/release build	Zero serial overhead in release	✓ Confirmed via binary size reduction

[Comparison table to be extended with backend, web dashboard, and AI validation results.]

6.6 Error Analysis

6.6.1 Embedded Subsystem

Three categories of error were identified and analysed:

Sensor read errors. The DHT22 occasionally returns NaN on the first read after power-up due to its required 1-second warm-up delay. The NaN guard in `updateSensors()` discards the invalid reading silently. No invalid values were written to the database in any test session.

Records lost during outage (open). Records generated during a Wi-Fi outage are discarded. The system does not implement local buffering or retry-on-reconnect.

[Error analysis for the backend, web dashboard, and AI subsystems to be added.]

7

Discussion

7.1 Validity of Results

7.1.1 Embedded Subsystem

The sensor measurements are consistent with the manufacturer-specified accuracy of the DHT22 ($\pm 0.5^\circ\text{C}$, $\pm 2\text{--}5\%$ RH) and within the expected operational range of the MAX30102 for resting-state adult subjects. The BPM standard deviation of 2.3 BPM reflects the inherent variability of beat-to-beat interval detection under minor motion artefacts. The SpO_2 accuracy of $\pm 1\%$ is within the $\pm 2\%$ tolerance generally accepted.

[Backend results validity discussion to be added.]

[Web dashboard and AI results validity discussion to be added.]

7.2 Limitations

7.2.1 Embedded Subsystem

No local offline buffer. Records generated during network outages are discarded.

Plaintext NVS storage. Firebase credentials (API key, email, password) are stored in NVS without encryption. The ESP32 NVS partition supports native encryption via an NVS key partition; this should be enabled before any production deployment.

Implicit shared-memory atomicity. The sensor globals are 32-bit floats written on Core 1 and read on Core 0 without explicit synchronisation. A FreeRTOS mutex or atomic qualifier is the correct long-term solution.

Single patient per device. The current design maps one device to exactly one patient and one room. Multi-patient rooms would require either multiple devices or a structural change to the data model.

7.2.2 Backend Subsystem

[To be completed by the backend team.]

7.2.3 Web Dashboard Subsystem

[To be completed by the web dashboard team.]

7.2.4 AI Subsystem

[To be completed by the AI team.]

7.3 ESP32 Constraints Impact

The most significant platform constraint encountered was the non-thread-safety of the mbedTLS context, which imposed the non-obvious architectural requirement of deferring Firebase initialisation until the task is running on Core 0.

The 8 KiB task stack was found to be adequate for the TLS handshake and async send operations, with an estimated 1.5 KiB headroom.

The choice of string-based JSON construction introduces heap fragmentation over long sessions.

7.4 Robustness

The unified `wifiConnect()` function using `WiFi.begin()` handles both cold-start failure and mid-session dropout correctly, as confirmed by Session 2 and Session 3 of the stability tests.

The delta-threshold filter provides robustness against spurious sensor noise: fluctuations smaller than 0.3 °C, 0.5 % RH, or 2 % SpO₂ do not generate database writes, reducing both network load and the volume of low-information records in the database.

7.5 Identified Improvements

Priority improvements for future versions of the embedded subsystem:

1. **Local offline buffer.** A LittleFS-backed FIFO queue for records generated during network outages, with automatic retry on reconnection. This would eliminate the record loss observed in Sessions 2 and 3.
2. **NVS encryption.** Enable the ESP32 native NVS encryption partition to protect credentials at rest.
3. **Thread-safe sensor globals.** Replace raw float globals with a mutex-protected struct or a FreeRTOS queue for safe inter-core communication.
4. **Wi-Fi modem sleep.** Implement modem sleep between upload intervals to reduce average current usage for future battery-powered deployments.

[Improvements for the backend, web dashboard, and AI subsystems to be added.]

8

Conclusion

8.1 Summary

This project designed, implemented, and validated a complete ESP32-based embedded acquisition node for a hospital room and patient monitoring system. The firmware acquires temperature, humidity, heart rate, and blood oxygen saturation from two sensor peripherals, applies local signal processing to derive clinically meaningful values, and transmits authenticated records to Firebase Realtime Database at a near-real-time rate of up to 1 Hz.

The system is structured around four integrated subsystems — the ESP32 embedded node, the backend server, the web dashboard, and the AI prediction layer — each developed independently and connected through a shared Firebase Realtime Database. This report focused on the embedded subsystem contribution and provided placeholder sections for the remaining three subsystems.

8.2 Results Summary

The key results achieved by the embedded subsystem are:

- Sensor accuracy within manufacturer specifications: $\pm 0.1^\circ\text{C}$ for temperature, $\pm 0.4\%$ for humidity, $\pm 1.0\text{ BPM}$ for heart rate, and $\pm 1.0\%$ for SpO_2 .

- Steady-state end-to-end latency of $\approx 2\,200$ ms for room data (dominated by the DHT22 hardware sampling period) and ≈ 250 ms for patient data.
- Successful automatic Wi-Fi recovery in both mid-session dropout and cold-start failure scenarios, with reconnection within 10–15 s.
- Stable email/password Firebase authentication with full event traceability via the async callback, confirmed across all test sessions.
- Zero invalid records written to the database across all sessions, thanks to the NaN guard and IR threshold checks.
- Complete configuration persistence across five cold reboots via NVS.

[Backend results summary to be added.]

[Web dashboard results summary to be added.]

[AI results summary to be added.]

8.3 Limitations

The primary limitation of the embedded subsystem is the absence of a local offline buffer: records generated during network outages are permanently lost. Secondary limitations include plaintext NVS credential storage, implicit float atomicity assumptions in the inter-core shared globals, and the absence of a device-side RTC for accurate timestamping during outages.

[Limitations for all other subsystems to be summarised here.]

8.4 Future Work

The following directions are identified for future development of the complete system:

1. **Multi-room scalability** (backend + embedded): extend the data model and backend to support concurrent streams from multiple ESP32 nodes deployed across different hospital rooms.

2. **Real-time push notifications** (backend + web): replace the polling-based dashboard with WebSocket or Server-Sent Events for sub-second alert delivery to medical staff.
3. **Enhanced AI model** (AI): extend the prediction model to incorporate multi-parameter correlation (e.g. combined BPM, SpO₂, and room temperature trends) to improve anomaly detection sensitivity and reduce false positive rates.
4. **Motion artefact rejection** (embedded): add an accelerometer to suppress SpO₂ readings during patient movement.
5. **Power management** (embedded): implement Wi-Fi modem sleep to reduce power consumption for potential battery-backup deployments.

9

AI Evaluation